

Helix Thready — API Request Collections (curl + HTTPie)

Table of Contents

1. What this is
2. Setup
3. Collections (10 endpoint groups)
4. Conventions used in every request
5. Ops smoke test
6. Maturity & fail-loud reminders
7. Provenance & open items

Rev	Date	Author	Change
1	2026-07-22	swarm (API & SDKs — materials)	Initial per-endpoint-group collections (10 groups × curl + HTTPie) grounded in openapi.yaml

Table of Contents

1. What this is
2. Setup
3. Collections (10 endpoint groups)
4. Conventions used in every request
5. Ops smoke test
6. Maturity & fail-loud reminders
7. Provenance & open items

1. What this is

Copy-pasteable request collections for every REST endpoint group of the `/v1` surface, in **two forms per group**:

- `*.curl.sh` — executable Bash scripts of `curl` invocations (one commented block per operation).
- `*.http` — **HTTPie CLI** command collections (`http ...` lines using HTTPie's `key=value` / `key:=raw` / `==query` / `Header:value` syntax).

Every request is derived directly from `../../openapi.yaml` — paths, methods, request bodies, path/query params, the `Idempotency-Key` header, and the `hmacAuth` callback signature. Example payloads reuse the spec's own `examples` where present (login, search hybrid, MeTube/Boba callbacks, event-sink registration).

`[VERIFIED]` all `*.curl.sh` pass `bash -n`. `[DEFAULT — adjustable]` all ids/tokens are placeholders in `env.example`; no request was executed against a live server.

2. Setup

```
cd docs/public/research/mvp/api/materials/examples
cp env.example .env          # then edit THREADY_TOKEN / THREADY_BASE / ids
source .env

bash auth.curl.sh            # run a whole group...
# ...or copy a single block. HTTPie collections are sourced the same way:
source .env && head -n 20 auth.http # then paste an `http ...` line
```

`env.example` sets `THREADY_BASE` (defaults to the **dev** server `https://dev.thready.hxd3v.com/v1`), `THREADY_TOKEN` (a JWT access token **or** a scoped API key `sk-...`), resource ids, a generated `Idempotency-Key`, and the per-provider `THREADY_HMAC_SECRET` used to sign inbound callbacks.

3. Collections (10 endpoint groups)

Group	Files	Operations covered (from openapi.yaml)
auth	<code>auth.curl.sh</code> · <code>auth.http</code>	login, refresh, logout, me, TOTP enroll/verify, oauth2 authorize, api-keys CRUD, JWKS
accounts	<code>accounts.curl.sh</code> · <code>accounts.http</code>	accounts CRUD, branding PUT, account users list/invite, users get/patch (accounts+users tags)
channels	<code>channels.curl.sh</code> · <code>channels.http</code>	messengers list, channels list/register/get/delete, sync (poll + backfill)
posts	<code>posts.curl.sh</code> · <code>posts.http</code>	posts list/filter/cursor, get, thread, post assets
processing	<code>processing.curl.sh</code> · <code>processing.http</code>	process, reprocess, processing state, jobs list, HMAC-signed provider callbacks
assets	<code>assets.curl.sh</code> · <code>assets.http</code>	assets list/get, content (302 + Range 206), redownload, post assets, downloads
search	<code>search.curl.sh</code> · <code>search.http</code>	POST /search — hybrid, semantic+filters, keyword
skills	<code>skills.curl.sh</code> · <code>skills.http</code>	skills list/register (edges), get
billing	<code>billing.curl.sh</code> · <code>billing.http</code>	plans, subscription get/put, usage (current + named period)
events	<code>events.curl.sh</code> · <code>events.http</code>	catalog, sticky snapshot, event-sinks CRUD, SSE stream + WebSocket note

`GET /messengers` (its own tag) is folded into **channels** as it precedes channel registration; the **ops** endpoints (`/healthz` , `/readyz` , `/version`) are in §5 below.

4. Conventions used in every request

- **Auth** — `Authorization: Bearer $THREADY_TOKEN` (bearerAuth/apiKeyAuth). Unauthenticated ops (`login` , `refresh` , `jwtks` , `healthz` , `readyz` , `oauth2/authorize`) carry no header, matching `security: []` in the spec.
- **Idempotency** — unsafe async POSTs send `Idempotency-Key: $THREADY_IDEMPOTENCY_KEY` ; a same-key + different-body replay returns `409 conflict` .
- **Inbound callbacks** — `POST /processing/callbacks/{provider}` is signed with `X-Thready-Signature: sha256=<hmac>` over the **raw body** (the curl script computes it with `openssl dgst -sha256 -hmac`), **not** a JWT. Idempotent on `job_id` .
- **Pagination** — list ops take `?limit=` (1-200, default 50) and `?cursor=` ; iterate on `meta.next_cursor` .
- **Ranged content** — `GET /assets/{id}/content` follows a `302` to a signed URL or streams with `Range: → 206` .

5. Ops smoke test

Unauthenticated liveness/readiness + the authenticated version/contract identity:

```
curl -sS "$THREADY_BASE/healthz"      # 200 alive
curl -sS -i "$THREADY_BASE/readyz"    # 200 ready / 503 unavailable
curl -sS "$THREADY_BASE/version" -H "Authorization: Bearer $THREADY_TOKEN" # api_version,
                                     contract_hash...
```

6. Maturity & fail-loud reminders

The spec annotates non-GA operations with `x-thready-maturity` ; the collections echo these in comments so a caller is not surprised by a `503` :

- **build_new** (may `503 unavailable`): `channels/sync` backfill, `posts/process` + `reprocess` , `assets/redownload` , `downloads` , `search` , all `events` / `event-sinks` .
- **fail-loud**: `POST /search` returns `503` if the non-semantic **HashEmbedder** stub is the active provider — the response `embedder` field echoes the real provider (`HELIX_EMBEDDING_PROVIDER=llama`).
[GAP: #1 HelixLLM]
- **foundation**: `channels` register, `processing` callbacks, `skills` .
- **design**: `billing` subscription/usage.

7. Provenance & open items

- `[VERIFIED]` request shapes, headers, params and example payloads trace 1:1 to `../.. /openapi.yaml`; curl scripts pass `bash -n`.
 - `[OPEN: ex-1]` WebSocket (`/v1/events/ws`) is **not** curl/HTTPIe-able (it is a WS upgrade); the `events` collection shows the SSE stream (curl-able) and a `websocat` note for WS, both specified in `../.. /event-bus-contract.md` §5.
 - `[OPEN: ex-2]` No request was run against a live/staging server here; treat these as ready-to-run templates, not a green integration run.
-

Made with love ♥ by Helix Development.